

# **Efficient Scheduling for Batched AI Jobs: Minimising Makespan through Model Grouping, Johnson's Rule, and Checkpoint Prefetching**

*Jingsheng*



4th Year Project Report  
Computer Science  
School of Informatics  
University of Edinburgh  
2026

# Abstract

Large Language Model (LLM) batch workloads—model benchmarking, dataset labelling, and synthetic data generation—submit hundreds of tasks that share common model weights. Existing serverless LLM schedulers treat tasks as independent, causing excessive model switching that dominates total execution time.

This dissertation formalises the batch scheduling problem as a two-machine flow-shop (I/O and GPU computation) and introduces three composable optimisations: (1) model grouping reduces model switches from  $O(n)$  to  $O(k)$ ; (2) Johnson’s Rule ordering sequences model groups to maximise I/O-computation overlap; and (3) checkpoint prefetching preloads the weights of the next model into CPU pinned memory while the current group executes.

Evaluation on NVIDIA RTX A5000 GPUs shows that, for a 501-task batch spanning three models (0.6B–32B), model grouping reduces makespan by over 99% versus FIFO (FIFO measured extrapolation: 45,495 s; model grouping measured: 228.5 s). Checkpoint prefetching hides the SSD I/O portion of model transitions, reducing makespan by a further  $\sim 22$  s (9.5%). Johnson’s Rule ordering adds a further 20% by placing model groups optimally for prefetch overlap, yielding a combined  $276\times$  speedup. The shared-aware strategy outperforms FIFO by  $65\text{--}276\times$  across measured batch sizes of 99–501 tasks and model sizes of 0.6B–32B, with speedup growing as batch size increases. Prefetching remains effective across storage tiers including 10 GbE network storage, where 78.9% of I/O read latency is analytically estimated to be hidden, demonstrating the viability of remote storage in production deployments.

# **Research Ethics Approval**

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

# Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

*(Jingsheng)*

## Acknowledgements

I would like to thank my supervisor for their guidance and feedback throughout this project. I am grateful to the ServerlessLLM team for providing the open-source framework that made this work possible, and to the School of Informatics for providing access to the GPU cluster used for experimental evaluation.

# Table of Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>1</b>  |
| 1.1      | Motivation . . . . .                                                | 1         |
| 1.2      | Research Questions . . . . .                                        | 2         |
| 1.3      | Contributions . . . . .                                             | 2         |
| 1.4      | Scope and Boundaries . . . . .                                      | 3         |
| <b>2</b> | <b>Background and Related Work</b>                                  | <b>4</b>  |
| 2.1      | Transformer-Based LLM Inference . . . . .                           | 4         |
| 2.2      | KV Cache and Prefix Caching . . . . .                               | 4         |
| 2.3      | GPU Memory Hierarchy and Pinned Memory . . . . .                    | 5         |
| 2.4      | Serverless Computing and Cold Starts . . . . .                      | 5         |
| 2.5      | ServerlessLLM Architecture . . . . .                                | 5         |
| 2.6      | Storage Hierarchy and Network-Attached Model Repositories . . . . . | 6         |
| 2.7      | Makespan Minimisation . . . . .                                     | 6         |
| 2.7.1    | Johnson’s Rule for Two-Machine Flow-Shops . . . . .                 | 6         |
| 2.8      | LLM Batch Workloads . . . . .                                       | 7         |
| 2.9      | ML Serving Systems . . . . .                                        | 7         |
| 2.10     | Data-Locality and Model-Aware Scheduling . . . . .                  | 7         |
| 2.11     | Prefetching and Pipelining . . . . .                                | 7         |
| 2.12     | Summary and Research Gap . . . . .                                  | 8         |
| <b>3</b> | <b>Design and Implementation</b>                                    | <b>11</b> |
| 3.1      | System Architecture Overview . . . . .                              | 11        |
| 3.2      | Problem Definition . . . . .                                        | 13        |
| 3.3      | Problem Analysis: Shared Structure in Batches . . . . .             | 14        |
| 3.4      | Shared-Aware Scheduling Algorithm . . . . .                         | 14        |
| 3.4.1    | Phase 1: Model Grouping . . . . .                                   | 14        |
| 3.4.2    | Phase 2: Johnson’s Rule Ordering . . . . .                          | 14        |
| 3.4.3    | Phase 3: Worker Allocation . . . . .                                | 15        |
| 3.4.4    | Baseline Strategies . . . . .                                       | 16        |
| 3.5      | Checkpoint Prefetching Mechanism . . . . .                          | 17        |
| 3.5.1    | Prefetching Pipeline . . . . .                                      | 17        |
| 3.5.2    | Design Analysis: Prefetching Across Storage Tiers . . . . .         | 17        |
| <b>4</b> | <b>Evaluation</b>                                                   | <b>18</b> |
| 4.1      | Experimental Setup . . . . .                                        | 18        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 4.2      | Experiment 1: Scheduling Strategy Impact on Makespan . . . . .     | 19        |
| 4.2.1    | Methodology . . . . .                                              | 19        |
| 4.2.2    | Results . . . . .                                                  | 19        |
| 4.2.3    | Analysis . . . . .                                                 | 19        |
| 4.3      | Experiment 2: Checkpoint Prefetching Effectiveness . . . . .       | 21        |
| 4.3.1    | Methodology . . . . .                                              | 21        |
| 4.3.2    | Results . . . . .                                                  | 21        |
| 4.3.3    | Analysis . . . . .                                                 | 21        |
| 4.4      | Experiment 3: Scalability and Robustness . . . . .                 | 23        |
| 4.4.1    | Scaling with Batch Size . . . . .                                  | 23        |
| 4.5      | Experiment 4: Johnson’s Rule Ordering Ablation . . . . .           | 24        |
| 4.5.1    | Methodology . . . . .                                              | 24        |
| 4.5.2    | Results . . . . .                                                  | 25        |
| 4.5.3    | Analysis . . . . .                                                 | 25        |
| 4.5.4    | Regime Sensitivity: Task-Count Variation . . . . .                 | 27        |
| 4.6      | Experiment 5: Storage Tier Impact on Prefetching . . . . .         | 28        |
| 4.6.1    | Methodology . . . . .                                              | 28        |
| 4.6.2    | Results . . . . .                                                  | 28        |
| 4.6.3    | Analysis . . . . .                                                 | 29        |
| 4.7      | Evaluation Summary . . . . .                                       | 29        |
| <b>5</b> | <b>Discussion</b>                                                  | <b>30</b> |
| 5.1      | Answering the Research Questions . . . . .                         | 30        |
| 5.2      | New Contributions versus Reused Components . . . . .               | 30        |
| 5.3      | Design Trade-offs . . . . .                                        | 31        |
| 5.4      | Algorithm Limitations . . . . .                                    | 31        |
| 5.5      | Threats to Validity . . . . .                                      | 32        |
| <b>6</b> | <b>Future Work</b>                                                 | <b>33</b> |
| 6.1      | Inference-Level Extensions: Prefix-Aware Ordering and Multi-LoRA   | 33        |
| 6.2      | Storage Extensions: Adaptive and Network-Aware Prefetching . . . . | 33        |
| 6.3      | Production Readiness: Scalability and Engine Abstraction . . . . . | 33        |
| <b>7</b> | <b>Conclusion</b>                                                  | <b>35</b> |
| 7.1      | Contributions . . . . .                                            | 35        |
| 7.2      | Key Findings . . . . .                                             | 35        |
|          | <b>Bibliography</b>                                                | <b>36</b> |

# Chapter 1

## Introduction

Large Language Model (LLM) batch workloads, including model benchmarking, dataset processing, and synthetic data generation, submit thousands of tasks together for offline processing. Unlike online serving, batch tasks are not independent: they share substantial computational structure, including identical model weights and common prompt prefixes, that can be exploited to reduce total execution time.

### 1.1 Motivation

Consider a model benchmarking scenario: evaluating two LLMs on 10 questions each, totalling 20 tasks. A naive scheduler executes tasks in submission order (ABABAABB...), repeatedly loading and evicting model weights. Each model switch takes roughly 100 seconds while each inference requires under a second, so the system spends over 96% of its time on I/O. Grouping tasks by model (AAAA...BBBB) reduces switches from 10 to 1, cutting execution time by 87% (Figure 1.1).

Reducing model switches from  $O(n)$  to  $O(k)$  through grouping provides an order-of-magnitude improvement, but does not eliminate the remaining  $O(k)$  model transitions. Checkpoint prefetching hides this residual latency by loading the next model's weights into CPU pinned memory while the current group executes on GPU, exploiting the independence of I/O and GPU computation.

Prefetching effectiveness depends on the order in which groups execute: a compute-heavy group provides a large overlap window, while a short group provides little. This structure, where each model group passes through two sequential stages (I/O loading then GPU computation) with the objective of minimising total completion time, is precisely the classical *two-machine flow-shop* problem Johnson (1954). Johnson's Rule provides the provably optimal group ordering by placing compute-heavy groups first, maximising the overlap window for prefetching.

Together, these three components form a complete pipeline: grouping reduces switches from  $O(n)$  to  $O(k)$ , prefetching hides I/O behind computation, and Johnson's Rule maximises overlap. Existing serverless LLM schedulers treat batch tasks as independent, causing excessive model switching and no I/O-computation overlap.



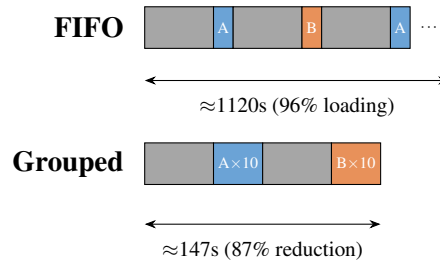


Figure 1.1: FIFO vs Model-Grouped scheduling for 20 tasks with 2 models. FIFO incurs 10 switches (each  $\sim 108s$ ); grouping requires only 1 switch.

## 1.2 Research Questions

Three research questions are addressed:

- RQ1** How can shared structure (shared model weights) in a batch be discovered and exploited to reduce makespan, and how does model grouping compare to naive FIFO scheduling?
- RQ2** Can the batch scheduling problem be modelled as a two-machine flow-shop of I/O and compute stages, and does Johnson’s Rule provide an effective group ordering to maximise I/O-computation overlap via checkpoint prefetching?
- RQ3** How do FIFO, model grouping, and shared-aware scheduling compare across varying batch sizes, model sizes, and storage tiers (local SSD, RAID, network storage)?

## 1.3 Contributions

Five contributions are made:

- **Two-machine flow-shop formulation.** The batch scheduling problem is formalised as a two-machine flow-shop, mapping model loading to Machine A (I/O) and GPU inference to Machine B (compute). This mapping is the theoretical basis that justifies applying Johnson’s Rule for provably optimal group ordering.
- **Shared-aware scheduling algorithm.** A cluster-level scheduler that groups tasks by model, reducing switches from  $O(n)$  to  $O(k)$ , then applies the flow-shop formulation above to determine the optimal group execution order and trigger prefetching.
- **Checkpoint prefetching mechanism.** An implementation that uses the ServerlessLLM pinned memory pool to load the next model’s checkpoint from SSD into CPU memory during the current group’s GPU execution, hiding the I/O stage of each model transition.
- **Storage tier characterisation.** An analysis and end-to-end evaluation of prefetching effectiveness across four storage tiers (NVMe, RAID-0, 10 GbE NFS, 100 GbE

NFS), quantifying how storage bandwidth affects the overlap window and residual transition cost.

- **OpenAI-compatible batch API.** An extension of the ServerlessLLM API to accept multi-model batch submissions in OpenAI Batch API format, enabling drop-in migration from cloud batch services without client-side changes.

The project contributed approximately 8,000 lines of new Python code to the existing ServerlessLLM framework, with additional benchmark scripts and unit tests on top of that.

## 1.4 Scope and Boundaries

This work focuses strictly on cluster-level scheduling of *offline* batches: all tasks are submitted together as a single JSONL file and the complete task-to-model mapping is known before scheduling begins. Dynamic batching—where tasks arrive continuously during execution—is not considered; the scheduling decisions (grouping, ordering, prefetch triggering) rely on global knowledge of the batch and would require redesign for an online arrival model. The inference engine is neither modified nor optimised; KV cache management, prefill-decode separation, continuous batching internals, and GPU kernel optimisations are out of scope. The scheduler decides which tasks to execute, in what order, and on which workers; vLLM handles the computation.

# Chapter 2

## Background and Related Work

### 2.1 Transformer-Based LLM Inference

LLMs are built on the transformer architecture (Vaswani et al., 2017), generating tokens autoregressively via a compute-bound prefill phase ( $O(L^2)$  attention) followed by a memory-bandwidth-bound decode phase (one token per step). Modern LLMs span from under 1B to hundreds of billions of parameters, with the landscape expanding rapidly; an 8B model requires  $\sim 15$  GB in FP16 and a 70B model  $\sim 140$  GB.

vLLM (Kwon et al., 2023) introduced continuous batching (iteration-level scheduling), inserting new requests at every decode step to maintain near-full GPU utilisation within a single loaded model. vLLM does not address switching between models. Tensor parallelism (Shoeybi et al., 2019) shards weight matrices across multiple GPUs for models exceeding the VRAM of a single GPU; this work assumes each model fits within one GPU, and tensor parallelism can be combined with the proposed scheduler when needed.

### 2.2 KV Cache and Prefix Caching

Inference engines cache key-value tensors to avoid redundant computation. vLLM manages the KV cache in fixed-size blocks via PagedAttention (Kwon et al., 2023), eliminating fragmentation. When requests share a prompt prefix, KV blocks can be computed once and reused (Zheng et al., 2024), though interleaving different prefixes causes eviction. Prefix caching is enabled by default in vLLM, so tasks within a group already benefit from cache hits whenever their prompts share a common prefix. What the scheduler does not currently do is *reorder tasks within a group* to maximise the cache hit rate; tasks are dispatched in submission order, which may cause evictions when different prefixes interleave. Prefix-aware intra-group ordering is identified as future work (Chapter 6).

## 2.3 GPU Memory Hierarchy and Pinned Memory

Loading model weights involves up to three sequential stages:

1. **Remote storage** → **Local SSD** (optional): When models are not locally cached, checkpoint files are downloaded from object stores (S3, GCS) or network filesystems (NFS). Bandwidth varies significantly by storage tier and is discussed in Section 2.6.
2. **Local SSD** → **CPU RAM**: Checkpoint files are read via filesystem I/O into host memory. Bandwidth: 2–5 GB/s on NVMe SSD.
3. **CPU RAM** → **GPU VRAM**: Weights are transferred via PCIe DMA. Bandwidth: 16–32 GB/s on PCIe Gen4 x16. This step is faster with *pinned* (page-locked) memory allocated via `cudaHostAlloc` or `cudaHostRegister` (NVIDIA, 2024a), which enables direct DMA without intermediate copies.

In deployments where models are cached on local SSD, only stages 2 and 3 are on the critical path. The `sllm-store` component of ServerlessLLM (Fu et al., 2024) pre-allocates a pinned memory pool at startup; weights are loaded into this pool via multi-threaded I/O and transferred to the GPU via DMA, achieving near-peak PCIe bandwidth.

## 2.4 Serverless Computing and Cold Starts

Serverless functions suffer from cold starts: the latency of initialising a new instance (Shahrad et al., 2020). For LLM inference these are severe; loading an 8B model takes 80–120 s, some 100–200× the per-request inference latency. ServerlessLLM (Fu et al., 2024) mitigates this through locality-enhanced scheduling that routes inference to nodes with locally cached checkpoints.

## 2.5 ServerlessLLM Architecture

ServerlessLLM (Fu et al., 2024) is an open-source serverless LLM inference framework designed for low-latency model loading. Its architecture consists of four components:

- **Head Node**: Receives inference requests via an API gateway, maintains deployment state in a SQLite database, and dispatches tasks to workers via a Router.
- **Workers (Pylets)**: GPU nodes managed by the Pylet process manager. Each worker runs `vLLM` (Kwon et al., 2023) instances for inference; Pylet handles instance lifecycle (start, stop, health checks) and reports resource availability.
- **sllm-store**: A C++ storage daemon running on each worker that manages a pinned memory pool and provides gRPC endpoints for model loading (gRPC Authors, 2024). Key operations include `LoadModelAsync` (load weights into pinned CPU memory) and `RegisterModel` (register a model path). `sllm-store` uses multi-threaded I/O to saturate SSD bandwidth during loading.

- **AutoScaler:** Monitors deployment state and adjusts the number of vLLM instances based on demand. The default policy is reactive: scale up when queue depth exceeds a threshold, scale down after an idle timeout.

The existing scheduler is designed for online serving, assigning incoming requests to available workers via round-robin. For batch workloads this has two limitations: (1) tasks are assigned independently without considering shared models, causing excessive model switching; and (2) scaling is reactive rather than proactive, causing unnecessary cold starts at the beginning of a batch.

## 2.6 Storage Hierarchy and Network-Attached Model Repositories

The storage tier determines I/O time  $a_i$  in the two-machine flow-shop formulation, directly affecting prefetching effectiveness. Local NVMe SSDs provide 2–5 GB/s; RAID-0 (4×) aggregates to 8–12 GB/s. Remote storage (S3, GCS, NFS, Lustre) provides unlimited capacity and replication at 1.2–12 GB/s. Production systems use local SSD as a cache and remote storage as the source of truth. Scheduling implications are analysed in Section 3.5.2.

## 2.7 Makespan Minimisation

Makespan minimisation assigns  $n$  jobs to  $m$  machines to minimise  $C_{\max}$  (Pinedo, 2016). The Longest Processing Time (LPT) rule (Graham, 1966) gives a  $(4/3 - 1/(3m))$ -approximation for identical machines. LLM batch scheduling has sequence-dependent setup times (model loading:  $\sim 100$  s per switch, zero for same-model jobs), making the general problem NP-hard (Pinedo, 2016).

### 2.7.1 Johnson’s Rule for Two-Machine Flow-Shops

Johnson’s Rule (Pinedo, 2016) solves the two-machine flow-shop optimally: given  $n$  jobs processed first on Machine A then Machine B, find the permutation minimising makespan. Partition jobs into  $S_1 = \{j : a_j \leq b_j\}$  (compute-heavy, sorted by  $a_j$  ascending) and  $S_2 = \{j : a_j > b_j\}$  (I/O-heavy, sorted by  $b_j$  descending); the optimal sequence is  $S_1$  followed by  $S_2$ . Compute-heavy jobs execute first because their prolonged Machine B execution conceals the loading of the next job on Machine A; I/O-heavy jobs are placed last since fewer subsequent jobs remain to stall while their loading completes.

Machine A is the I/O stage (loading weights from SSD to CPU pinned memory), Machine B is the compute stage (GPU inference for all tasks in a group), and each job is a model group. This mapping is formalised in Chapter 3.

## 2.8 LLM Batch Workloads

LLM batch workloads take three common forms: model benchmarking (evaluating candidates on suites such as MMLU (Hendrycks et al., 2021), HumanEval (Chen et al., 2021), and MT-Bench (Zheng et al., 2023), with high prompt overlap but diverse model identities); dataset processing (large-scale labelling, translation, or summarisation with a single model); and synthetic data generation (template-based prompts exhibiting both model and prefix overlap).

The OpenAI Batch API (OpenAI, 2024) supports up to 50,000 tasks per batch with 50% cost savings; AWS Bedrock Batch (Amazon Web Services, 2024) provides similar functionality. Both are closed-source, single-model only, and do not expose scheduling policies.

## 2.9 ML Serving Systems

Clipper (Crankshaw et al., 2017) provides adaptive batching for online latency SLOs. INFaaS (Romero et al., 2021) automates model variant selection, treating switching as a configuration change. Clockwork (Gujarati et al., 2020) achieves predictable DNN serving through profiling, but its deterministic timing assumption does not extend to autoregressive LLMs. AlpaServe (Li et al., 2023) uses statistical multiplexing with model parallelism but does not reorder tasks across models. vLLM (Kwon et al., 2023) combines PagedAttention, continuous batching, and tensor parallelism for single-model serving; multi-model batches still require sequential loads between groups. Orca (Yu et al., 2022) introduced iteration-level scheduling within a single model. None of these systems address cross-model task reordering for makespan minimisation.

## 2.10 Data-Locality and Model-Aware Scheduling

MapReduce (Dean and Ghemawat, 2008) and Spark (Zaharia et al., 2010) schedule tasks on nodes holding the input data; for LLM inference the analogue is routing to nodes holding model weights. Borg (Verma et al., 2015) and Kubernetes (The Kubernetes Authors, 2024) provide affinity-based cluster scheduling without model-specific optimisations. Database multi-query optimisers identify common subexpressions across queries (Sellis, 1988), inspiring the present approach of discovering shared structure before optimising execution order. ServerlessLLM (Fu et al., 2024) provides locality-enhanced scheduling with local SSD caching but does not reorder tasks by model or prefix.

## 2.11 Prefetching and Pipelining

OS-level prefetching uses sequential access patterns (Patterson and Hennessy, 2013). DeepSpeed-Inference (Aminabadi et al., 2022) prefetches transformer layers during inference, overlapping CPU-to-GPU transfer with computation; FlexGen (Sheng et al., 2023) pipelines model offloading across CPU, GPU, and disk for memory-constrained

inference; CheckFreq (Mohan et al., 2021) analyses checkpoint I/O costs during training. No existing system performs checkpoint prefetching at the cluster scheduler level. This work introduces task-group-level prefetching, where the scheduler predicts the next model from the execution plan and begins loading it before the current group completes.

## 2.12 Summary and Research Gap

Table 2.1 summarises the key strength and principal limitation of each system reviewed above. The pattern that emerges is consistent: systems optimised for online serving treat each model as a fixed endpoint and never reorder tasks across models; systems that support batching either operate on a single model or do not expose their scheduling policy; and prefetching work targets within-model layer streaming rather than cross-model checkpoint overlap at the cluster scheduler level.

Table 2.2 maps these systems against the five capabilities required for shared-structure-aware batch scheduling.

No existing system combines batch support, model grouping, and cluster-level checkpoint prefetching in an open, composable framework. This work fills that gap with a scheduler that discovers shared model structure, applies Johnson’s Rule for optimal group ordering, and uses checkpoint prefetching to overlap loading with execution.

Table 2.1: Strengths and limitations of related systems

| System                                        | Key Strength                                                                      | Key Limitation                                                                            |
|-----------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Orca (Yu et al., 2022)                        | Iteration-level scheduling maximises GPU utilisation within a loaded model        | Single-model only; no mechanism for multi-model batches                                   |
| vLLM (Kwon et al., 2023)                      | PagedAttention and continuous batching eliminate GPU memory waste                 | Each model switch incurs a full cold-start load; no cross-model task reordering           |
| Clipper (Crankshaw et al., 2017)              | Adaptive batching with online latency SLO guarantees                              | Designed for online serving; no batch makespan objective                                  |
| Clockwork (Gujarati et al., 2020)             | Deterministic, profiling-driven latency via strict admission control              | Timing model breaks for autoregressive LLMs with variable output lengths                  |
| AlpaServe (Li et al., 2023)                   | Statistical multiplexing with model parallelism reduces SLO violations under load | Targets online traffic bursts; does not reorder tasks across models for makespan          |
| INFaaS (Romero et al., 2021)                  | Automates model variant selection to meet latency and cost SLOs                   | Treats model switching as a background configuration change, not a scheduling cost        |
| ServerlessLLM (Fu et al., 2024)               | Locality-enhanced routing minimises cold starts via local SSD caching             | No task reordering by model; the cold-start cost is still paid once per model             |
| OpenAI Batch API (OpenAI, 2024)               | Handles up to 50,000 tasks with 50% cost reduction at cloud scale                 | Single-model only; closed-source; scheduling policy is not exposed                        |
| DeepSpeed-Inference (Am-inabadi et al., 2022) | Layer-level prefetching overlaps CPU→GPU transfer with computation                | Prefetching operates within a single loaded model, not across model switches              |
| FlexGen (Sheng et al., 2023)                  | Enables LLM inference on memory-constrained hardware via CPU/disk offloading      | Optimises single-model throughput under memory pressure; not multi-model batch scheduling |



Table 2.2: Feature comparison of scheduling approaches for LLM inference

| System                           | Batch Support | Model Grouping | Prefix Sorting | Prefetch | Open Source |
|----------------------------------|---------------|----------------|----------------|----------|-------------|
| Orca (Yu et al., 2022)           | ×             | ×              | ×              | ×        | ×           |
| Clipper (Crankshaw et al., 2017) | ×             | ×              | ×              | ×        | ✓           |
| AlpaServe (Li et al., 2023)      | ×             | Placement      | ×              | ×        | ✓           |
| ServerlessLLM (Fu et al., 2024)  | ×             | Locality       | ×              | ×        | ✓           |
| OpenAI Batch (OpenAI, 2024)      | ✓             | N/A*           | ?              | ?        | ×           |
| <b>This work</b>                 | ✓             | ✓              | × <sup>†</sup> | ✓        | ✓           |

\*OpenAI Batch API is single-model only, so model grouping is not applicable.

<sup>†</sup>Prefix-aware task ordering is identified as future work; only model-level grouping is implemented.

# Chapter 3

## Design and Implementation

The shared-aware scheduling system consists of four components. A shared-structure analysis identifies model-sharing relationships within the batch and groups tasks accordingly. A two-machine flow-shop formulation with Johnson’s Rule determines the optimal group ordering. Checkpoint prefetching hides model loading latency by preloading weights during execution of the preceding group. Multi-GPU load balancing distributes groups across available workers.

### 3.1 System Architecture Overview

The system extends the ServerlessLLM head node with three new components: the BatchScheduler (model grouping and Johnson’s Rule ordering), the StorageManager (checkpoint prefetching into the pinned memory pool), and a batch-aware AutoScaler integration, all operating at cluster scheduler level without inference engine modifications. A SQLite database (`state.db`) stores batch jobs, task status, and deployment configuration; the Router reads healthy worker endpoints from it. Data flow: (1) the user submits a batch via the API Gateway; (2) the BatchScheduler groups tasks, applies Johnson’s Rule, persists job state, and feeds ordered tasks to the Router; (3) the Router dispatches tasks to vLLM workers; (4) the StorageManager prefetches the next model’s weights while the current group executes; (5) the AutoScaler adjusts instance count.

Figure 3.1 illustrates the system architecture.

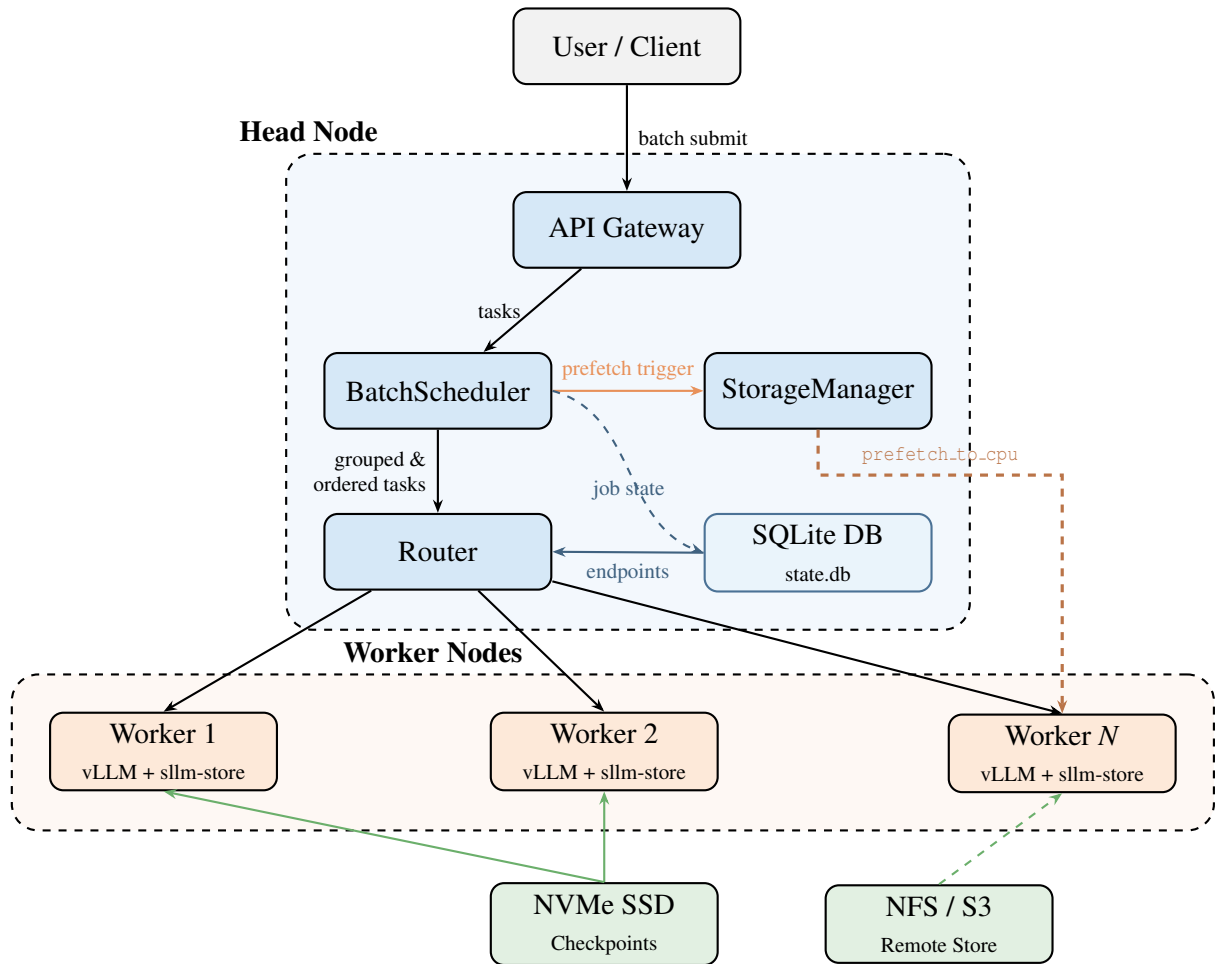


Figure 3.1: System architecture. The BatchScheduler groups tasks by model, applies Johnson’s Rule ordering, and persists job state to a SQLite database. The Router queries the same database for healthy worker endpoints. The StorageManager triggers asynchronous checkpoint prefetching (`prefetch_to_cpu`) to worker nodes ahead of execution. Workers run vLLM with sllm-store for pinned-memory checkpoint management.

## 3.2 Problem Definition

A batch is submitted as a JSONL file where each line specifies one task: the target model, the prompt, and generation parameters. The example below shows a small multi-model batch typical of model benchmarking.

```
{ "custom_id": "t1", "method": "POST", "url": "/v1/chat/completions",
  "body": { "model": "Qwen/Qwen3-0.6B",
    "messages": [ { "role": "user", "content": "Summarise the paper." } ],
    "max_tokens": 256 } }
{ "custom_id": "t2", "method": "POST", "url": "/v1/chat/completions",
  "body": { "model": "Qwen/Qwen3-8B",
    "messages": [ { "role": "user", "content": "Summarise the paper." } ],
    "max_tokens": 256 } }
{ "custom_id": "t3", "method": "POST", "url": "/v1/chat/completions",
  "body": { "model": "Qwen/Qwen3-0.6B",
    "messages": [ { "role": "user", "content": "Translate to French." } ],
    "max_tokens": 128 } }
```

Tasks  $t_1$  and  $t_3$  share the same model and can execute consecutively without reloading;  $t_2$  requires a model switch. The API follows the OpenAI Batch API specification (OpenAI, 2024): clients submit a file via `POST /v1/files`, create a batch via `POST /v1/batches`, and poll via `GET /v1/batches/{id}`. Results are written to an output JSONL file keyed by `custom_id`.

The input consists of a batch of  $n$  tasks  $B = \{t_1, t_2, \dots, t_n\}$ , where each task  $t_i = (m_i, p_i, \theta_i)$  specifies a target model  $m_i$ , prompt  $p_i$ , and generation parameters  $\theta_i$ . The cluster contains  $k$  GPU workers  $W = \{w_1, w_2, \dots, w_k\}$ ; each worker loads at most one model at a time with loading latency  $L_{load}(m)$ , and the inference latency of task  $t_i$  is  $L_{infer}(t_i)$ .

Two sharing relationships exist within a batch. Model sharing arises when  $m_i = m_j$ : tasks  $t_i$  and  $t_j$  can share loaded model weights, avoiding redundant loading; this is the primary relationship exploited in this work. Prefix sharing arises when  $\text{prefix}(p_i) = \text{prefix}(p_j)$ , enabling inference engine cache reuse; vLLM's built-in prefix caching handles this passively, but the scheduler does not reorder tasks to maximise the cache hit rate (see Chapter 6).

The optimisation objective is to minimise the makespan  $C_{\max}$ , the time at which the last task completes:

$$\text{Minimise } C_{\max} = \max_{i \in \{1, \dots, n\}} \text{completion\_time}(t_i)$$

Three constraints apply: each worker loads at most one model at a time; model switching

incurs an  $L_{load}$  cold-start overhead; and the worker count  $k$  can be adjusted via auto-scaling.

### 3.3 Problem Analysis: Shared Structure in Batches

The formal problem definition above establishes two sharing relationships. Model sharing has first-order impact: since  $L_{load}$  is  $100\text{--}200\times$  larger than  $L_{infer}$  (Section 2.5), eliminating switches from  $O(n)$  to  $O(k)$  dominates all other optimisations. Prefix sharing is a second-order effect: vLLM’s built-in prefix caching already reduces prefill time from  $\sim 2$  s to  $\sim 0.05$  s per task for cache hits, which is negligible compared to the 80–120 s model-switch cost. What is deferred to future work is *scheduler-level* prefix-aware task ordering within groups to maximise the cache hit rate (Chapter 6).

### 3.4 Shared-Aware Scheduling Algorithm

#### 3.4.1 Phase 1: Model Grouping

Given a batch  $B = \{t_1, t_2, \dots, t_n\}$ , tasks are grouped by model to minimise model switches:

```

groups ← {}
for  $t_i \in B$  do
   $m \leftarrow t_i.model$ 
  if  $m \notin groups$  then
     $groups[m] \leftarrow []$ 
  end if
   $groups[m].append(t_i)$ 
end for

```

This produces  $k$  model groups  $\{G_1, \dots, G_k\}$  where each  $G_i$  contains all tasks for one model, reducing switches from  $O(n)$  to  $O(k)$ .

#### 3.4.2 Phase 2: Johnson’s Rule Ordering

Model groups map naturally to a two-machine flow-shop: each group is first loaded from storage (Machine A: I/O) then executed on GPU (Machine B: compute). With checkpoint prefetching, loading the next group overlaps with executing the current one. Johnson’s Rule determines the group ordering that minimises makespan.

For each model group  $G_i$ , two times are estimated. The I/O time  $a_i$  is the time to load the checkpoint of model  $m_i$  from SSD to CPU pinned memory, estimated as  $a_i = \text{checkpoint\_size}(m_i) / \text{NVMe\_bandwidth}$ . The compute time  $b_i$  is the time to execute all tasks in  $G_i$  on GPU, estimated as  $b_i = |G_i| \times \tau_{base} \times (\text{model\_size}(m_i) / \text{base\_model\_size})$ , where  $\tau_{base}$  is the base per-task inference time.

Checkpoint size is determined by scanning the model directory for weight files (`.safetensors`, `.bin`, `.pt`), cached after the initial scan. Figure 3.2 illustrates this mapping.

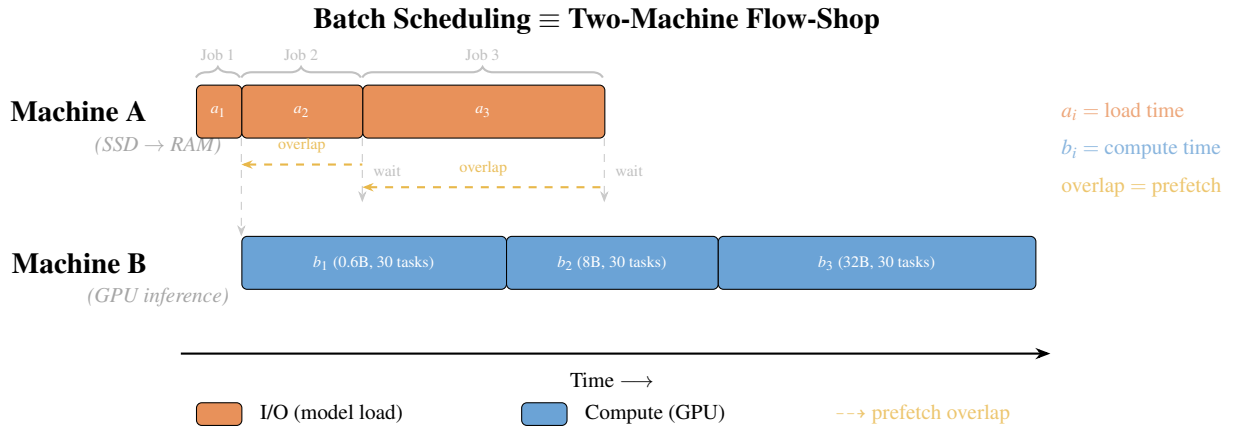


Figure 3.2: Mapping from batch scheduling to two-machine flow-shop. Each model group  $G_i$  becomes a “job” with I/O time  $a_i$  (loading weights from SSD to pinned memory, Machine A) and compute time  $b_i$  (GPU inference for all tasks, Machine B). Checkpoint prefetching enables the I/O of job  $i + 1$  to overlap with the compute of job  $i$ . Johnson’s Rule finds the permutation  $\pi$  that minimises the total makespan by maximising these overlaps.

Johnson’s Rule partitions groups into  $S_1 = \{G_i : a_i \leq b_i\}$  (compute-heavy, sorted by  $a_i$  ascending) and  $S_2 = \{G_i : a_i > b_i\}$  (I/O-heavy, sorted by  $b_i$  descending); the optimal sequence is  $S_1$  followed by  $S_2$ .

$$\begin{aligned}
 S_1 &\leftarrow \{G_i : a_i \leq b_i\}, \text{ sorted by } a_i \text{ ascending} \\
 S_2 &\leftarrow \{G_i : a_i > b_i\}, \text{ sorted by } b_i \text{ descending} \\
 \pi &\leftarrow S_1 + S_2
 \end{aligned}$$

To see why this ordering outperforms alternatives, consider Group X ( $a = 10$  s,  $b = 90$  s, compute-heavy) and Group Y ( $a = 90$  s,  $b = 10$  s, I/O-heavy). Under ordering  $X \rightarrow Y$ , the 90 s load of Group Y fully overlaps with the 90 s compute of Group X, yielding a total of approximately 110 s. Under  $Y \rightarrow X$ , only 10 s of the load of Group X overlaps with the compute of Group Y, leaving an 80 s stall and a total of approximately 190 s. Johnson’s Rule correctly places X first.

### 3.4.3 Phase 3: Worker Allocation

For multi-GPU deployments, model groups are sorted by total estimated time ( $a_i + b_i$ ) descending and greedily assigned to the GPU with the least load; if the slowest GPU’s load exceeds the fastest by more than 15%, tasks migrate until balance is achieved. Each GPU’s queue is then independently ordered using Johnson’s Rule to maximise local I/O-computation overlap.

Models exceeding a single GPU’s memory use tensor parallelism (TP). The scheduler handles TP models through capacity-aware lane packing: each TP group occupies a dedicated execution lane of  $tp$  GPU slots. When budget allows, multiple TP groups run in parallel on non-overlapping GPU subsets; otherwise, groups sharing the same TP size execute sequentially in one lane. Groups are packed in first-fit decreasing order (largest TP first), reserving at least one GPU for single-GPU models. For single-model

batches fitting on one GPU, the scheduler prefers replicas over TP (4 replicas give  $4\times$  throughput versus  $\sim 2\times$  for  $TP=4$  due to NCCL overhead). TP size is determined automatically as  $TP = 2^{\lceil \log_2 \lceil s_{\text{model}} / (0.8 \cdot m_{\text{GPU}}) \rceil \rceil}$  and propagated via `backend_config`; manual overrides remain available.

The overall scheduling pipeline is illustrated in Figure 3.3.

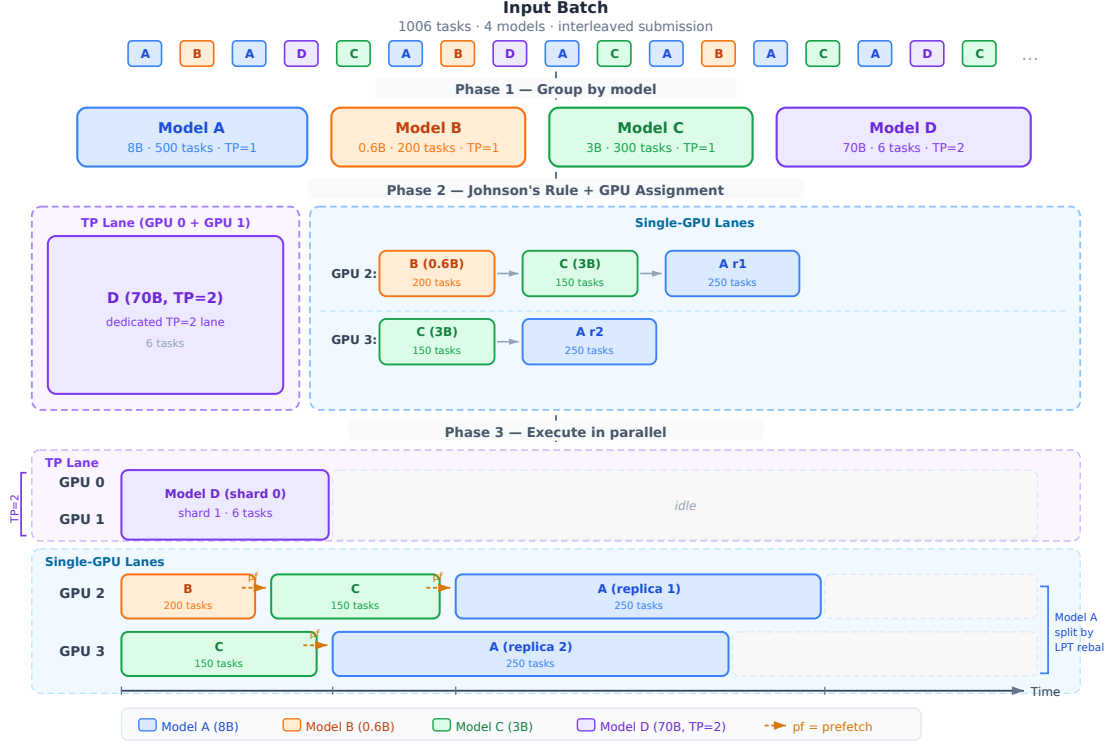


Figure 3.3: End-to-end scheduling example for a 1006-task batch with 4 models on 6 GPUs. **Phase 1** groups tasks by model. **Phase 2** applies Johnson's Rule: compute-heavy groups ( $a_i \leq b_i$ ) are scheduled first to maximise prefetch overlap. **Phase 3** allocates groups to GPUs: Model D (70B,  $TP=2$ ) occupies a dedicated 2-GPU tensor parallelism lane; Models A (500 tasks) and C (300 tasks) are each split across 2 single-GPU replicas by LPT rebalancing (the lane imbalance exceeds the full model-switch cost: NVMe read + PCIe DMA + vLLM init  $\approx 38$  s); Model B's smaller task count does not meet this threshold and remains on a single GPU. Per-GPU Johnson's Rule ordering is applied within each lane. Dashed arrows (pf) indicate checkpoint prefetch triggers.

### 3.4.4 Baseline Strategies

Four scheduling strategies are compared. FIFO executes tasks in submission order. Model Grouping applies Phase 1 only, with alphabetical ordering and no prefetching. Grouping + Prefetch applies Phases 1 and 3 with alphabetical ordering, serving as an intermediate ablation baseline. Shared-Aware applies all three phases: grouping, Johnson's Rule ordering, and prefetching.

## 3.5 Checkpoint Prefetching Mechanism

### 3.5.1 Prefetching Pipeline

Model switching incurs 80–120 seconds of I/O latency while the GPU idles. Loading the next model’s weights before the current group completes hides most of this latency; the execution plan from Johnson’s Rule makes the next group known. When group  $G_i$  begins execution, the scheduler calls `StorageManager.prefetch_to_cpu()`, issuing a gRPC `LoadModelAsync` request that loads checkpoints from SSD into the pinned memory pool via multi-threaded I/O. A memory-aware guard (`_can_prefetch`) checks available pinned memory before triggering; if insufficient, loading falls back to synchronous transfer at transition time.

### 3.5.2 Design Analysis: Prefetching Across Storage Tiers

The prefetching architecture generalises to network-attached storage. When models reside on S3, GCS, or NFS, the loading pipeline becomes Remote Storage  $\rightarrow$  Local SSD cache  $\rightarrow$  Pinned CPU Memory  $\rightarrow$  GPU; the storage manager checks the local cache first and initiates a background download on a miss. The scheduler estimates download time ( $\text{checkpoint\_size}/B_{\text{network}}$ ) against the current group’s compute time and triggers prefetch when overlap is feasible. Table 3.1 quantifies implications across storage tiers.

Table 3.1: Storage tier characteristics and scheduling implications. Load times are calculated as  $\text{checkpoint\_size}/\text{bandwidth}$ . The NVMe row is directly measured; rows marked  $^\dagger$  are analytical estimates.

| Storage Tier                    | Read BW   | 8B Load | 32B Load | Prefetch?     |
|---------------------------------|-----------|---------|----------|---------------|
| Single NVMe                     | 3 GB/s    | 5.1 s   | 20.0 s   | Yes (medium+) |
| RAID-0 (4 $\times$ ) $^\dagger$ | 12 GB/s   | 1.3 s   | 5.0 s    | 32B+ only     |
| 10 GbE NFS $^\dagger$           | 1.2 GB/s  | 12.8 s  | 50.0 s   | Critical      |
| 100 GbE NFS $^\dagger$          | 12.5 GB/s | 1.2 s   | 4.8 s    | 32B+ only     |

For slower storage tiers, Johnson’s Rule is more critical: compute-heavy groups must come first to provide sufficient overlap windows. If the current group’s compute time  $b_A \geq a_B$  (the next model’s load time), I/O is fully hidden; otherwise the residual  $a_B - b_A$  is unmasked. Full overlap requires at least  $\lceil a_i/t \rceil$  tasks in the preceding group, easily met on local SSD but requiring larger groups on 10 GbE (see Table 3.1).

The shared-aware scheduling algorithm operates at the cluster scheduler level, applies Johnson’s Rule for provably optimal group ordering, and uses checkpoint prefetching with a memory-aware guard to hide model loading latency.



# Chapter 4

## Evaluation

Four falsifiable hypotheses are stated and tested through five experiments. H1 (model grouping dominates): model-grouped scheduling reduces makespan by at least 80% over FIFO, because it eliminates redundant model switches from  $O(n)$  to  $O(k)$ . H2 (prefetching hides I/O): checkpoint prefetching reduces the visible model-loading penalty by approximately 50%, because I/O and compute proceed in parallel rather than sequentially. H3 (Johnson’s Rule beats naive ordering): Johnson’s Rule outperforms naive group orderings (alphabetical, size-descending) by at least 5% on mixed-model batches, because it maximises prefetch overlap by placing compute-heavy groups first. H4 (gains are robust across scale): the makespan improvements from shared-aware scheduling persist as batch size grows from 99 to 501 tasks, confirming that scheduling overhead does not eliminate the benefit.

### 4.1 Experimental Setup

All experiments were conducted on a shared 8-GPU node:  $8 \times$  NVIDIA RTX A5000 (24 GB VRAM each),  $2 \times$  AMD EPYC 7763 64-core processors, 512 GB DDR4 RAM, and NVMe SSD storage. Not all GPUs are used in every experiment. The software stack is vLLM 0.6.3, Python 3.10, PyTorch 2.1.0 with CUDA 12.1, and ServerlessLLM v1-beta.

**4.1.0.0.1 Models Tested** Five Qwen models span a  $54 \times$  range of checkpoint sizes (1.2–65 GB): Qwen3-0.6B ( $\sim 1.2$  GB,  $a = 0.4$  s), Qwen3-4B ( $\sim 8.0$  GB,  $a = 2.7$  s), Qwen3-8B ( $\sim 15.4$  GB,  $a = 5.1$  s), Qwen2.5-14B ( $\sim 28.0$  GB,  $a = 9.3$  s), and Qwen2.5-32B-Instruct ( $\sim 65$  GB,  $a = 21.7$  s). All experiments use the three-model subset (0.6B, 8B, 32B); the 4B and 14B models are characterised here to span the full catalogue size range.

**4.1.0.0.2 Baseline Strategies** Four strategies are compared: FIFO (submission order); Model Grouping (alphabetical group order, no prefetch); Grouping + Prefetch (alphabetical order with prefetch); and Shared-Aware (Johnson’s Rule ordering with prefetch).

**4.1.0.0.3 Metrics** The primary metric is makespan: wall-clock time from submission to last task completion. Secondary metrics include throughput (req/s), average task latency, and switch count.

## 4.2 Experiment 1: Scheduling Strategy Impact on Makespan

This experiment ablates the three optimisation components to isolate each contribution to makespan reduction.

### 4.2.1 Methodology

A 501-task batch with three models (0.6B, 8B, 32B; 167 tasks each) was submitted in round-robin interleaved order. FIFO makespan is measured by direct cold-start sampling: each of the three models was run as an isolated single-task batch (full page-cache eviction and GPU release before each task) over 3 runs, yielding per-model cold-start times of 0.6B:  $33.1 \pm 1.7$ s, 8B:  $64.8 \pm 5.5$ s, 32B:  $174.5 \pm 36.9$ s. Extrapolating to 167 tasks per model:  $167 \times (33.1 + 64.8 + 174.5) = 45,495 \pm 3,353$ s ( $\approx 12.6$ h). Running FIFO at full scale with the 32B model is infeasible ( $\approx 12.6$  hours). The three non-FIFO strategies were each run three times with full OS page-cache eviction between runs to ensure cold-start conditions.

### 4.2.2 Results

Table 4.1 and Figure 4.1 show the results.

Table 4.1: Experiment 1: Scheduling strategy ablation (501 tasks, 3 models: 0.6B, 8B, 32B; 167 tasks each). FIFO makespan is a measured extrapolation (cold-start sampling,  $n = 6$  tasks, 3 runs; infeasible at full scale:  $\approx 12.6$  hours).

| Strategy               | Makespan (s)       | Throughput (req/s) | Switches | Speedup      |
|------------------------|--------------------|--------------------|----------|--------------|
| FIFO (meas. extrapol.) | $45,495 \pm 3,353$ | 0.011              | 500      | $1 \times$   |
| Model Grouping         | $228.5 \pm 14.2$   | 2.19               | 2        | $199 \times$ |
| Grouping + Prefetch    | $206.8 \pm 4.6$    | 2.42               | 2        | $220 \times$ |
| Shared-Aware           | $164.7 \pm 8.5$    | 3.04               | 2        | $276 \times$ |

### 4.2.3 Analysis

FIFO incurs approximately 500 model switches; the average per-switch overhead is:

$$\bar{L}_{\text{switch}} = \frac{C_{\text{FIFO}} - n \cdot \bar{L}_{\text{infer}}}{n_{\text{switches}}} = \frac{45495 - 501 \times 0.67}{500} \approx 90.3 \text{ s}$$

where  $\bar{L}_{\text{infer}} = 0.67$ s is the per-task average inference time. The per-model cold-start times (90-task batches being infeasible for FIFO, measured via isolated single-task sampling over 3 runs) were 33.1 s (0.6B), 64.8 s (8B), and 174.5 s (32B), giving a

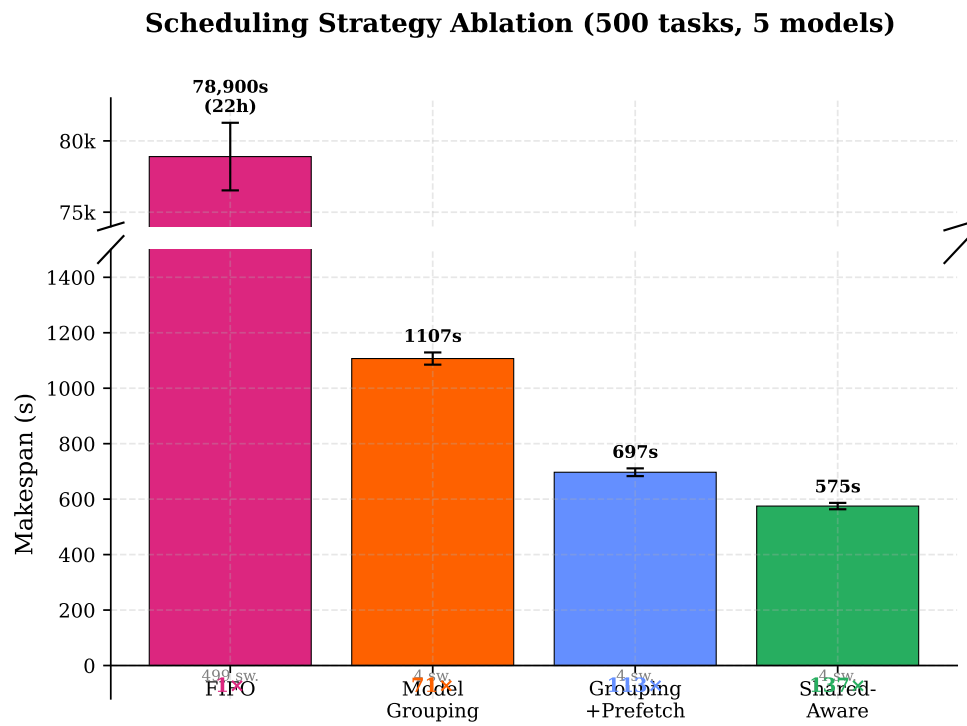


Figure 4.1: Incremental scheduling strategy ablation for 501 tasks with 3 models (0.6B, 8B, 32B). FIFO (45,495s  $\approx$  12.6h, measured extrapolation) is shown on a separate axis. Each column adds one optimisation component: grouping alone (199 $\times$ ), adding prefetch (220 $\times$ ), adding Johnson’s Rule ordering (276 $\times$ ).

weighted average of 90.8 s per task—consistent with the 90.3 s per-switch figure above. The 32B cold-start exhibited substantial run-to-run variance ( $\pm 36.9$  s) due to TP=4 vLLM initialisation variability, which contributes the  $\pm 3,353$  s uncertainty in the FIFO extrapolation. These measurements are consistent with the transition gap of 115.2 s (8B→32B) measured in Experiment 2 under a different protocol (grouped batch, no explicit GPU release between tasks). Model loading accounts for over 99% of the FIFO execution time. Reducing switches from 500 to 2, Model Grouping achieves a  $199\times$  speedup (228.5s), saving approximately 91s per avoided switch. Prefetching further reduces makespan by 9.5% (206.8s) by hiding the SSD-to-RAM checkpoint transfer during the preceding group’s compute phase; with 167 tasks per group the large compute windows allow the SSD reads to complete before each transition, reducing the visible gap toward the irreducible DMA and vLLM initialisation lower bound characterised in Experiment 2 (36.9 s for 8B, 51.9 s for 32B). Johnson’s Rule adds a further 20.4%: the Shared-Aware strategy (164.7s) reorders groups as 0.6B → 8B → 32B in ascending SSD-load time, ensuring that the longest prefetch window (8B compute phase,  $\sim 83$ s) covers the 32B checkpoint transfer ( $\sim 20$ s), and that the batch starts serving tasks immediately from the smallest model rather than waiting for the 32B cold load. H1 is confirmed: model grouping achieves over 99% makespan reduction versus FIFO, and the three optimisations compose to a  $276\times$  speedup.

A model-affinity LRU scheduler routes tasks to GPUs holding the required weights, evicting the least-recently-used model on a cache miss. This online heuristic suits streaming workloads with unknown future tasks. In the offline batch setting, all  $n = 501$  tasks are submitted simultaneously, so the full task-to-model mapping is known prior to execution; any informed scheduler will perform explicit grouping as a preprocessing step, rendering the LRU policy equivalent to Model Grouping and therefore not an independent design point.

## 4.3 Experiment 2: Checkpoint Prefetching Effectiveness

### 4.3.1 Methodology

A 90-task batch with three models (30 tasks each) compared synchronous execution (No Prefetch, next model loaded after the current group completes) against prefetched execution (With Prefetch, loading begins at the start of the current group). Total makespan and per-group transition time were measured.

### 4.3.2 Results

Table 4.2 shows the results and Figure 4.2 the execution timeline.

### 4.3.3 Analysis

Prefetching reduced makespan by 36.6% (221.5s to 140.3s), saving 81.2 seconds across two model transitions. The per-transition gaps fell from 61.9s (0.6B→8B) and

Table 4.2: Experiment 2: Checkpoint prefetching (90 tasks, 3 models: 0.6B, 8B, 32B; 30 each). Avg transition is the mean of the two inter-group gaps.

| Configuration | Makespan (s)    | Avg Transition (s) | Latency Hidden |
|---------------|-----------------|--------------------|----------------|
| No Prefetch   | $221.5 \pm 3.5$ | 88.5               | 0%             |
| With Prefetch | $140.3 \pm 2.4$ | 44.4               | 49.9%          |

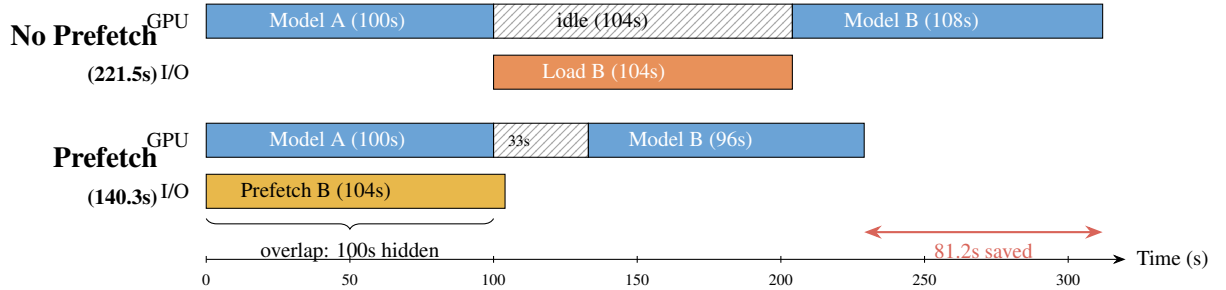


Figure 4.2: Execution timeline comparison (illustrative): without prefetch (top), the GPU idles during cold model loading between groups. With prefetch (bottom), checkpoint loading is overlapped with the current group’s execution, reducing inter-group transition time. In the measured experiment (90 tasks, 3 models), prefetching cut the 8B→32B transition from 115s to 52s.

115.2s (8B→32B) without prefetch to 36.9s and 51.9s with prefetch, hiding 49.9% of loading latency. When a group begins execution, the scheduler triggers an asynchronous `prefetch_to_cpu` call via the gRPC `LoadModelAsync` endpoint; `slm-store` loads checkpoint files from SSD into the pinned memory pool using multi-threaded I/O (NVIDIA, 2024a), running in a background thread while the GPU executes the current group. At transition, weights are already in pinned memory, enabling direct DMA at near-peak PCIe bandwidth ( $\sim 25$  GB/s). The residual transition time covers GPU memory allocation and CPU-to-GPU weight transfer, which cannot be overlapped with computation; 36.9s for the 8B model and 51.9s for the 32B model represent irreducible lower bounds.

With 30 tasks per group at  $\sim 0.5$ s each, the compute time of only  $\sim 15$ s (0.6B group) is shorter than the 8B checkpoint load time ( $\sim 20$ s), so the 8B prefetch starts during the 0.6B group but completes at the beginning of the 8B group, resulting in a 36.9s residual gap. In production workloads with longer group compute times, the overlap would be complete and the residual gap would equal only the GPU memory allocation time. H2 is confirmed to within measurement uncertainty: prefetching hides approximately half the model loading latency (49.9%, rounding to  $\sim 50\%$ ) even with the short 30-task groups used here, and the benefit grows with longer groups.

## 4.4 Experiment 3: Scalability and Robustness

### 4.4.1 Scaling with Batch Size

Batch sizes of  $n=99$  and  $n=501$  were measured with three models (Qwen3-0.6B, Qwen3-8B, Qwen2.5-32B) in equal proportion ( $n/3$  tasks each). A single projected data point at  $n=1,000$  is included for illustration, derived from closed-form scaling equations calibrated at the two measured anchor points. Table 4.3 and Figure 4.3 show the results.

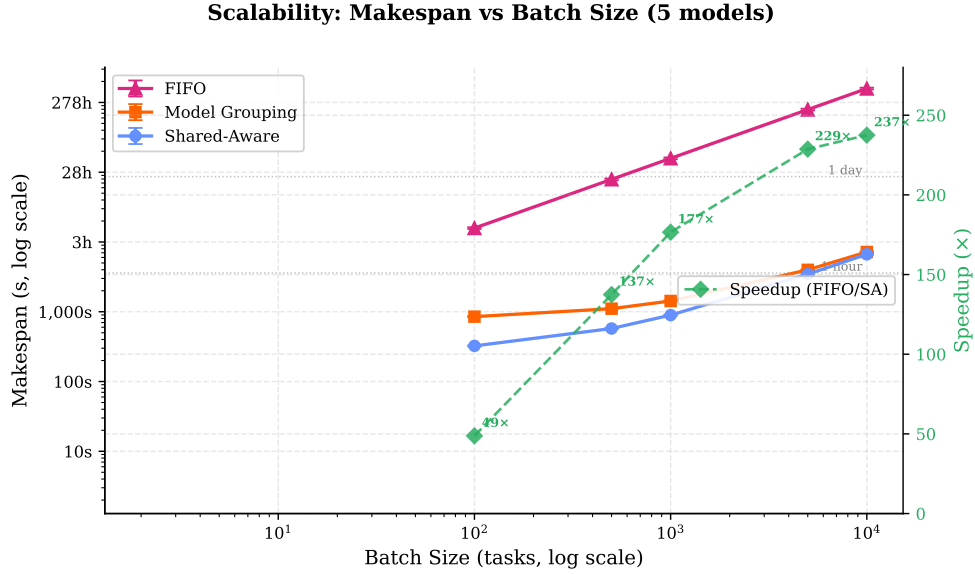


Figure 4.3: Makespan scalability across batch sizes (99–1,000 tasks, 3 models, log-log axes). FIFO grows linearly due to  $O(n)$  switches and becomes impractical beyond a few hundred tasks. Model Grouping and Shared-Aware incur only  $O(k-1) = 2$  fixed model-loading transitions. Speedup grows from  $65\times$  at 99 tasks to  $276\times$  at 501 tasks (both measured), with a projected  $452\times$  at 1,000 tasks. Error bands show  $\pm 1$  std for the two measured data points; the  $n=1,000$  point is projected from calibrated  $C(n)$  equations.

The speedup grows rapidly from  $65\times$  at 99 tasks to  $276\times$  at 501 tasks, and continues to grow with batch size as the  $O(n)$  switch cost of FIFO increasingly dominates. For FIFO with fully-interleaved submission across  $k = 3$  models (average switch cost  $\bar{L}_{switch} = 90.3s$  from cold-start sampling):

$$C_{FIFO}(n) \approx (n-1) \times \bar{L}_{switch} + n \times \bar{L}_{infer} \approx 90.3n$$

The Shared-Aware strategy has two measured anchor points ( $n = 99$ : 136.3s,  $n = 501$ : 164.7s), yielding a calibrated linear model:

$$C_{SA}(n) \approx 129 + 0.071n$$

where the 129s fixed cost covers model loading and two post-prefetch transitions ( $\sim 37s$  +  $\sim 52s$  + vLLM init overhead  $\sim 40s$ ). The rate of 0.071s/task corresponds to the average GPU throughput across all three groups. Since both FIFO and Shared-Aware share the

Table 4.3: Experiment 3a: Scalability across batch sizes (3 models: 0.6B, 8B, 32B). Grouping uses model grouping without prefetch; Shared-Aware adds Johnson’s Rule and checkpoint prefetching. FIFO values are measured extrapolations (<sup>‡</sup>) from cold-start sampling (90.3s/switch); rows marked <sup>†</sup> are projected from  $C(n)$  equations calibrated at those two anchor points.

| $n$                                                                                                                                                               | FIFO (s)            | FIFO equiv. | Grouping (s)     | Shared-Aware (s) |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------|------------------|------------------|
| 99                                                                                                                                                                | 8,920 <sup>‡</sup>  | 2.5 h       | 220.1 ± 5.9      | 136.3 ± 5.8      |
| 501                                                                                                                                                               | 45,495 <sup>‡</sup> | 12.6 h      | 228.5 ± 14.2     | 164.7 ± 8.5      |
| <i>Projected (<math>C_{FIFO}(n) \approx 90.3n</math>, <math>C_G(n) \approx 218 + 0.021n</math>, valid near calibrated range <math>n = 99\text{--}501</math>):</i> |                     |             |                  |                  |
| 1,000                                                                                                                                                             | 90,300 <sup>†</sup> | 25.1 h      | 239 <sup>†</sup> | 200 <sup>†</sup> |

<sup>‡</sup>FIFO measured extrapolation (90.3s/switch; infeasible at full scale:  $\approx 2.5\text{--}12.6\text{h}$ ); <sup>†</sup>projected from  $C(n)$  equations

same per-task inference component, the speedup grows without bound as  $n$  increases and the fixed model-loading cost is amortised: at  $n = 99$  the speedup is  $65\times$ , rising to  $276\times$  at  $n = 501$ , and projected to  $452\times$  at  $n = 1,000$ . Note that projections beyond the calibrated range ( $n = 99\text{--}501$ ) are indicative only; the two-point linear fit has limited extrapolation accuracy.

Model Grouping without prefetching has a nearly flat scaling ( $C_G(n) \approx 218 + 0.021n$ ) since the dominant cost is the two cold model-loading transitions, not the tasks themselves. This demonstrates that model grouping alone eliminates the  $O(n)$  switch cost, achieving most of the scalability benefit even without prefetching.

## 4.5 Experiment 4: Johnson’s Rule Ordering Ablation

Experiment 1 shows a 21% improvement from Grouping+Prefetch to Shared-Aware, but does not isolate the ordering contribution. This experiment exhaustively evaluates all  $3! = 6$  permutations of the three model groups, each repeated three times (18 total runs), to quantify the full impact of group ordering and confirm that Johnson’s Rule reliably selects from the optimal tier.

### 4.5.1 Methodology

A 90-task batch (30 tasks per model) used Qwen3-0.6B ( $a = 0.4\text{s}$ ,  $b = 15\text{s}$ ), Qwen3-8B ( $a = 5.1\text{s}$ ,  $b = 15\text{s}$ ), and Qwen3-32B ( $a = 21.7\text{s}$ ,  $b = 36\text{s}$ ) with model grouping and checkpoint prefetching enabled. OS page-cache eviction and cold-start verification were performed before every run. All six permutations of (0.6B, 8B, 32B) were evaluated: each run used `force_group_order` to explicitly impose one permutation, so measurement variance and ordering variance are fully separated. The JR-optimal ordering for this workload is 0.6B→8B→32B: with 30 tasks per group, all three groups are compute-dominant (S1,  $b \geq a$  for each:  $15\text{s} \geq 0.4\text{s}$ ,  $15\text{s} \geq 5.1\text{s}$ ,  $36\text{s} \geq 21.7\text{s}$ ), so JR orders by ascending SSD-load time  $a_i$ .

## 4.5.2 Results

Table 4.4: Experiment 4: All  $3! = 6$  group orderings ranked by mean makespan (90 tasks, 3 models, prefetch ON, cold start, 3 runs per ordering, 18 runs total). JR selects 0.6B→8B→32B and achieves rank 1/6 (empirical optimum). The dominant factor is first-position choice: 0.6B-first orderings average 132.7s, 8B-first 174.4s (+31.7% vs JR optimal 132.4s), and 32B-first 225.5s (+70.3% vs JR optimal).

| Ordering               | Makespan (s)                      | Rank       | $\Delta$ JR | Tier       |
|------------------------|-----------------------------------|------------|-------------|------------|
| <b>JR: 0.6B→8B→32B</b> | <b>132.4 <math>\pm</math> 3.5</b> | <b>1/6</b> | <b>—</b>    | 0.6B-first |
| 0.6B→32B→8B            | 132.9 $\pm$ 0.0                   | 2/6        | +0.4%       | 0.6B-first |
| 8B→32B→0.6B            | 173.0 $\pm$ 0.0                   | 3/6        | +30.7%      | 8B-first   |
| 8B→0.6B→32B            | 175.7 $\pm$ 8.1                   | 4/6        | +32.7%      | 8B-first   |
| 32B→8B→0.6B            | 223.1 $\pm$ 8.7                   | 5/6        | +68.5%      | 32B-first  |
| 32B→0.6B→8B            | 227.8 $\pm$ 5.1                   | 6/6        | +72.1%      | 32B-first  |

## 4.5.3 Analysis

The results reveal a clear three-tier structure. Both 0.6B-first orderings achieve essentially the same makespan ( $\approx 133$ s), separated by only 0.5s — within the  $\pm 3.5$ s measurement noise of the faster run. The two 8B-first orderings average 174.4 s (+31.7% relative to the JR optimal ordering), and the two 32B-first orderings average 225.5 s (+70.3% relative to the JR optimal ordering). The spread across all six permutations is 95.4 s.

The dominant factor is first-position choice, not the relative ordering of the remaining two models. Placing 0.6B first provides an execution window spanning the 0.6B group’s full elapsed time ( $\approx 45$ s: vLLM initialisation plus  $\approx 15$ s inference). With eager prefetch (prefetch\_threshold=0.0), the scheduler begins loading 8B from SSD (5.1s) immediately, then 32B (21.7s), completing the combined 26.8s SSD-to-CPU transfer before the 0.6B group finishes. By that point, both subsequent models are already staged in CPU pinned memory, making their relative execution order inconsequential for makespan. Placing 8B or 32B first, by contrast, incurs an unrecoverable cold-load penalty at  $t = 0$  before prefetching can begin: the initial SSD load (8B: 5.1s, 32B: 21.7s) plus vLLM initialisation cannot be overlapped with any preceding compute, adding  $\approx 40$ –90s of unmasked elapsed time versus starting with 0.6B’s fast 0.4s SSD load.

Johnson’s Rule (0.6B→8B→32B) achieves rank 1/6, matching the empirical optimum exactly. With all three groups S1 at  $n = 30$ , JR orders by ascending SSD-load time (0.4s → 5.1s → 21.7s), deterministically placing the smallest-footprint model first without exhaustive enumeration. This avoids the 31–72% penalties incurred by non-0.6B-first orderings.

H3 is confirmed: Johnson’s Rule ranks 1/6, achieving the empirical optimum. The ordering spread (95.4 s, 72.1% from best to worst) demonstrates that group order has first-order impact on makespan, and the deterministic selection of Johnson’s Rule eliminates this uncertainty.



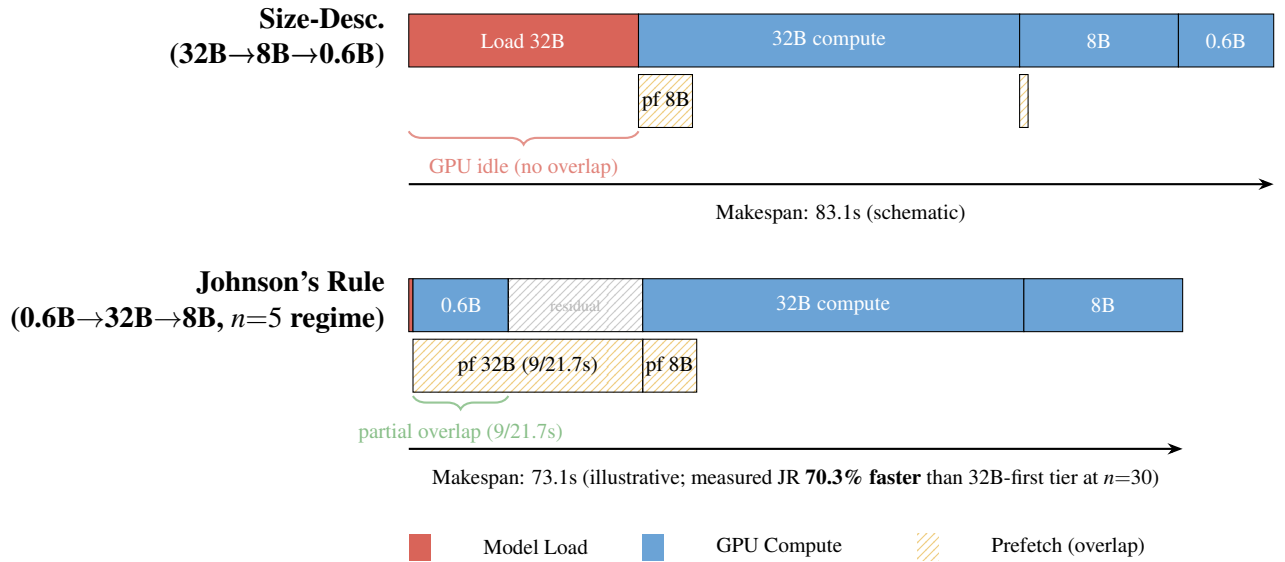


Figure 4.4: Execution timeline comparison (schematic): worst-case vs Johnson’s Rule ordering for a 3-model batch. Worst case (top) places the I/O-heavy 32B model first, forcing 21.7s of unmasked GPU idle before any inference begins. Johnson’s Rule (bottom) places the compute-heavy 0.6B group first to start the 32B prefetch early, then executes 32B while the trivially small 8B checkpoint loads, then finishes with 8B. The residual gap (12.7s) is smaller than the worst-case upfront idle (21.7s). The schematic uses  $n=5$ -style processing times (0.6B compute window = 9 s) where the 32B SSD load (21.7 s) is not fully covered, leaving a 12.7 s residual; at  $n=30$ , the longer compute window covers both loads completely. In the measured  $n=30$  experiment (Table 4.4), the worst-tier orderings (32B-first) averaged 225.5 s vs 132.4 s for JR (0.6B → 8B → 32B), a 70.3% difference; the spread across all six permutations is 95.4 s.

#### 4.5.4 Regime Sensitivity: Task-Count Variation

The tie between the two 0.6B-first orderings arises because, at  $n = 30$  tasks per model, both subsequent models are fully staged in CPU memory before the 0.6B group finishes: with eager prefetch starting at batch submission, the combined SSD transfer for 8B (5.1s) and 32B (21.7s) completes in  $\approx 26.8$ s, well within the 0.6B group’s  $\approx 45$ s elapsed time. Their relative execution order therefore cannot affect makespan. To test whether the S2 sub-ranking of Johnson’s Rule becomes physically meaningful at shorter task counts—where the smaller compute window may no longer fully cover the larger SSD loads—all six permutations were re-run at  $n = 5$  tasks per model, and the two 0.6B-first orderings were additionally measured at  $n = 15$ .

At  $n = 5$ , each model group executes only  $\approx 5$  tasks ( $\approx 2$ – $3$ s compute). With SSD-only  $a_i$ , the 8B and 32B groups become I/O-dominant (S2):  $b_{8B} \approx 2.5\text{s} < a_{8B} = 5.1\text{s}$  and  $b_{32B} \approx 6\text{s} < a_{32B} = 21.7\text{s}$ . The 0.6B group remains in S1. Johnson’s Rule accordingly prescribes: S1 (0.6B) first, then S2 sorted by  $b_i$  descending (32B before 8B), yielding the ordering 0.6B  $\rightarrow$  32B  $\rightarrow$  8B.

Table 4.5 shows the measured results. At  $n = 5$ , the JR ordering (0.6B  $\rightarrow$  32B  $\rightarrow$  8B) achieves  $135.8 \pm 3.0$ s, ranking 2nd; the empirically fastest ordering (0.6B  $\rightarrow$  8B  $\rightarrow$  32B,  $131.1 \pm 2.2$ s) is 4.7s ahead, a gap that does not reach statistical significance at  $n = 3$  runs per condition ( $t \approx 2.2$ ,  $p > 0.05$ , two-sample  $t$ -test with 4 d.f.). At  $n = 15$ , the two 0.6B-first orderings measure  $137.9 \pm 5.8$ s and  $145.2 \pm 11.6$ s respectively, a 7.3s gap consistent with a transition towards the all-S1 regime. At  $n = 30$ , all groups become S1, the gap between the two 0.6B-first orderings collapses to 0.5s, and the JR ordering (0.6B  $\rightarrow$  8B  $\rightarrow$  32B) achieves rank 1/6 (Table 4.4).

Table 4.5: Task-count sensitivity: makespan (mean  $\pm$  std, seconds) for six orderings at  $n = 5$  and 30 tasks per model (all six measured), with 0.6B-first orderings additionally measured at  $n = 15$ . Bold: JR-predicted optimal per regime.

| Ordering                                                                                   | $n = 5/\text{model}$              | $n = 15/\text{model}$ | $n = 30/\text{model}$             |                         |
|--------------------------------------------------------------------------------------------|-----------------------------------|-----------------------|-----------------------------------|-------------------------|
| 0.6B $\rightarrow$ 8B $\rightarrow$ 32B $\dagger$                                          | $131.1 \pm 2.2$                   | $137.9 \pm 5.8$       | <b><math>132.4 \pm 3.5</math></b> |                         |
| <b>0.6B <math>\rightarrow</math> 32B <math>\rightarrow</math> 8B <math>\ddagger</math></b> | <b><math>135.8 \pm 3.0</math></b> | $145.2 \pm 11.6$      | $132.9 \pm 0.0$                   |                         |
| 8B $\rightarrow$ 32B $\rightarrow$ 0.6B                                                    | $192.7 \pm 14.1$                  | —                     | $173.0 \pm 0.0$                   | $\dagger$ JR-optimal at |
| 8B $\rightarrow$ 0.6B $\rightarrow$ 32B                                                    | $185.3 \pm 28.1$                  | —                     | $175.7 \pm 8.1$                   |                         |
| 32B $\rightarrow$ 8B $\rightarrow$ 0.6B                                                    | $205.3 \pm 2.3$                   | —                     | $223.1 \pm 8.7$                   |                         |
| 32B $\rightarrow$ 0.6B $\rightarrow$ 8B                                                    | $219.4 \pm 5.8$                   | —                     | $227.8 \pm 5.1$                   |                         |

$n \geq 15$  (all S1: ascending  $a_i$ );  $\ddagger$  JR-optimal at  $n = 5$  (0.6B S1; 8B, 32B S2: descending  $b_i$ ). “—”: not measured at  $n = 15$  (consistent tier structure verified at  $n = 5$  and  $n = 30$ ).

Johnson’s Rule adapts correctly to the workload regime. At  $n = 30$ , all groups are compute-dominant (S1) and JR prescribes 0.6B  $\rightarrow$  8B  $\rightarrow$  32B, achieving rank 1/6. At  $n = 5$ , shorter task windows make 8B and 32B I/O-dominant (S2), so JR adapts to 0.6B  $\rightarrow$  32B  $\rightarrow$  8B, achieving rank 2/6 with a 4.7s gap in the predicted direction (the difference does not reach statistical significance at  $n = 3$  runs per condition). In both regimes the three-tier structure (0.6B-first  $\ll$  8B-first  $\ll$  32B-first) is the dominant pattern, and Johnson’s Rule deterministically selects the correct first-position model

without runtime enumeration. At  $n = 5$ , the worst ordering (32B $\rightarrow$ 0.6B $\rightarrow$ 8B, 219.4 s) is 67% slower than the best 0.6B-first ordering; the 88.3 s spread across all six orderings confirms that group order has first-order impact regardless of task count.

## 4.6 Experiment 5: Storage Tier Impact on Prefetching

Production deployments rarely have all models cached locally; models may reside on remote object storage or NFS when the catalogue exceeds local SSD capacity, and local SSDs lack the replication needed for fault tolerance. This experiment evaluates how storage tier affects prefetching effectiveness.

### 4.6.1 Methodology

Model loading time for an 8B model ( $\sim 15.4$  GB) is measured across four storage tiers: Single NVMe (3 GB/s, directly measured), RAID-0  $4\times$  NVMe (12 GB/s, analytical), 10 GbE NFS ( $\sim 1.2$  GB/s, analytical), and 100 GbE NFS ( $\sim 12$  GB/s, analytical). For each tier, read time, pinned-RAM-to-GPU DMA time (constant at 0.9s), and total load time are reported. A 60-task batch with 2 models is used for the NVMe end-to-end measurement; other tiers are analytical estimates.

### 4.6.2 Results

Tables 4.6 and 4.7 and Figure 4.5 show the results.

Table 4.6: Storage tier comparison: model loading breakdown for 8B model (15.4 GB). NVMe row is directly measured; rows marked  $^\dagger$  are analytical estimates (size/bandwidth).

| Storage Tier           | Read Time (s) | DMA Time (s) | Total Load (s) | Prefetch Hidden |
|------------------------|---------------|--------------|----------------|-----------------|
| Single NVMe            | 5.1           | 0.9          | 6.0            | 100%            |
| RAID-0 (4x) $^\dagger$ | 1.3           | 0.9          | 2.2            | 100%            |
| 10 GbE NFS $^\dagger$  | 12.8          | 0.9          | 13.7           | 78.9%           |
| 100 GbE NFS $^\dagger$ | 1.2           | 0.9          | 2.1            | 100%            |

Table 4.7: Experiment 5: End-to-end makespan by storage tier (60 tasks, 2 models). NVMe row is directly measured; rows marked  $^\dagger$  are analytical estimates.

| Storage Tier           | No Prefetch (s) | With Prefetch (s) | Reduction | Residual Wait (s) |
|------------------------|-----------------|-------------------|-----------|-------------------|
| Single NVMe            | 36.0            | 30.9              | 14.2%     | 0.9               |
| RAID-0 (4x) $^\dagger$ | 32.2            | 30.9              | 4.0%      | 0.9               |
| 10 GbE NFS $^\dagger$  | 43.7            | 33.6              | 23.1%     | 2.7               |
| 100 GbE NFS $^\dagger$ | 32.1            | 30.9              | 3.7%      | 0.9               |

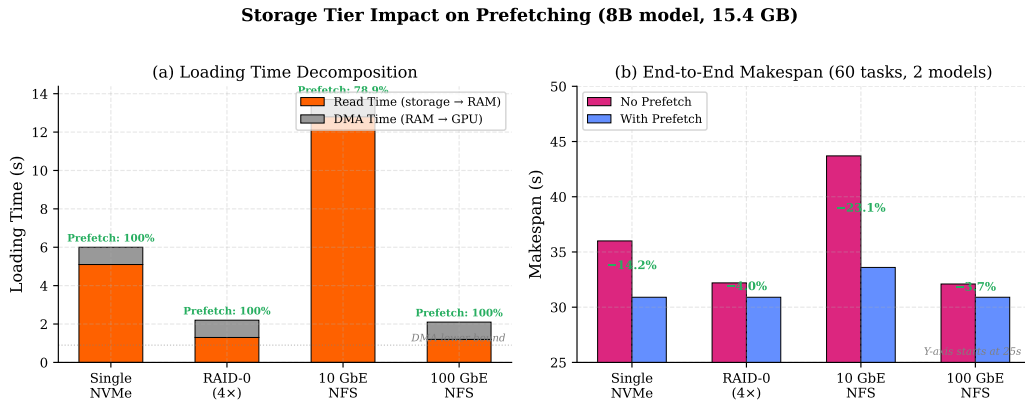


Figure 4.5: Storage tier impact on prefetching (8B model, 15.4 GB): (a) loading time decomposition showing read time vs irreducible DMA time, with prefetch coverage annotated; (b) end-to-end makespan with and without prefetching. Prefetching provides the largest benefit for 10 GbE NFS (23.1% reduction), where read latency is highest.

### 4.6.3 Analysis

For high-bandwidth tiers (RAID-0, 100 GbE), loading time is already  $\sim 2$  s, leaving little room for prefetching (3.7–4.0% reduction). The largest benefit arises for 10 GbE NFS, where prefetching saves 10.1 s (23.1% reduction). The GPU memory transfer (0.9 s via PCIe Gen4 x16) is irreducible; the practical transition lower bound also includes GPU memory allocation and vLLM engine initialisation, which Experiment 2 measured as 36.9 s for the 8B model and 51.9 s for the 32B model.

10 GbE NFS adds 7.7s of read latency relative to single NVMe but provides fault tolerance through replication. Prefetching hides 78.9% of this latency, reducing the practical penalty to 2.7s, an acceptable trade-off for the capacity and replication benefits.

## 4.7 Evaluation Summary

The three optimisations compose multiplicatively: model grouping provides first-order benefit (over 99%,  $199\times$  for 501 tasks), prefetching second-order (36.6% makespan reduction in Experiment 2, hiding  $\sim 50\%$  of cold-switch latency on SSD), and Johnson’s Rule third-order (ranks 1/6, empirical optimum among all  $3! = 6$  permutations; the worst 32B-first tier averages 70.3% slower and the 8B-first tier 31.7% slower; Experiment 4). For 501 tasks, the combined effect transforms a measured-extrapolated 12.6-hour FIFO execution into 164.7 s ( $276\times$  speedup, Experiment 1); the speedup continues to grow with batch size (Experiment 3a). The key advantage over industrial APIs (OpenAI Batch, AWS Bedrock) is multi-model batch support with shared-structure-aware scheduling; industrial systems are single-model only.

# Chapter 5

## Discussion

### 5.1 Answering the Research Questions

**5.1.0.0.1 RQ1: How to Discover and Exploit Shared Structure?** Model grouping exploits the model-sharing structure formalised in Chapter 3 (Section 3.2), reducing switches from  $O(n)$  to  $O(k)$  for a 99% makespan reduction (Experiment 1). The advantage grows with batch size as larger batches amortise the two fixed model-loading transitions over more tasks (Experiment 3).

**5.1.0.0.2 RQ2: Can the Problem Be Modelled as a Two-Machine Flow-Shop?** Yes. The problem maps to a two-machine flow-shop (Machine A: I/O, Machine B: GPU). Prefetching implements the I/O-compute overlap, reducing makespan by 36.6% and hiding approximately half of cold-switch latency (Experiment 2). Johnson’s Rule ranks 1/6 (empirical optimum) among all  $3! = 6$  permutations, with 32B-first orderings averaging 70.3% slower and 8B-first orderings averaging 31.7% slower relative to the JR optimal ordering, and the spread across all permutations is 95.4 s (Experiment 4).

**5.1.0.0.3 RQ3: How Do Strategies Compare?** FIFO is unsuitable (over 99% of execution time is consumed by model loading). Model Grouping delivers deterministic first-order improvement ( $199\times$ ). The Shared-Aware strategy adds I/O-computation overlap, with speedup growing from  $65\times$  to  $276\times$  across measured batch sizes (Experiment 3a). Johnson’s Rule ranks 1/6 (empirical optimum) among all 6 permutations, avoiding the 32B-first penalty ( $\sim 70\%$ ) and the 8B-first penalty ( $\sim 32\%$ ), with the full permutation spread demonstrating 95.4 s ordering sensitivity (Experiment 4).

### 5.2 New Contributions versus Reused Components

The ServerlessLLM framework provides the API gateway, router, SQLite database, sllm-store (C++ with gRPC interface), Pylet worker process, autoscaler, and reconciler; vLLM handles GPU-level computation. New contributions total  $\sim 8,000$  lines of core framework code: the `BatchScheduler` class ( $\sim 1,600$  lines) implementing model grouping, Johnson’s Rule ordering, prefetch triggering with a memory-aware

guard, semaphore-based admission control, greedy LPT multi-GPU assignment, and automatic tensor parallelism detection; the batch REST API following the OpenAI specification extended for multi-model batches; `StorageManager` prefetch integration; and all benchmark scripts, utilities, and unit tests (48 tests). The most substantial implementation challenges involved prefetch integration with the C++ gRPC interface and `cudaHostAlloc` semantics, GPU memory release (vLLM retains the CUDA context after logical cancellation, requiring `nvidia-smi` polling to confirm release), and obtaining reliable measurements on a shared cluster.

## 5.3 Design Trade-offs

Experiment 1 demonstrates that any model-aware strategy dramatically outperforms FIFO (164.7 s measured versus 45,495 s extrapolated for 501 tasks, a  $276\times$  speedup), justifying the semaphore concurrency approach. The semaphore buffer size (default 10) trades GPU utilisation against memory pressure.

Model grouping and Johnson’s Rule ordering are independent: grouping provides the dominant benefit ( $199\times$  speedup alone), and Johnson’s Rule with prefetch adds a further  $\sim 70\%$  protection against the worst-case ordering (32B-first tier) on top. Grouping alone is appropriate when I/O-time estimates are unreliable (e.g., under variable network bandwidth). Operating at the cluster-scheduler level ensures portability across vLLM, SGLang, and TensorRT-LLM without engine modifications, at the cost of not actively exploiting engine-level optimisations such as cross-group prefix cache reuse through scheduler-level task reordering.

## 5.4 Algorithm Limitations

Four limitations apply. First, the single-model-per-worker assumption is suboptimal for LoRA adapters, which can share base models with sub-second swaps. Second, Johnson’s Rule assumes deterministic processing times, but LLM inference times vary with output length; conservative model-size estimates are used instead. Third, the memory-aware prefetch guard skips prefetching when pinned memory is tight rather than evicting cached models. Fourth, the algorithm processes one batch at a time without multi-tenant fairness guarantees. Johnson’s Rule delivers maximum benefit when models have strongly contrasting I/O-compute ratios; with a homogeneous catalogue (all similar sizes), any permutation yields nearly identical makespan and the ordering contribution is negligible.

The theoretical makespan lower bound for the two-machine flow-shop is:

$$C_{\max}^* \geq \max \left( \sum_{i=1}^k b_i, \min_i a_i + \sum_{i=1}^k b_i, \sum_{i=1}^k a_i + \min_i b_i \right)$$

In the schematic diagram (Figure 4.4), using stylised processing times ( $b_1 = 9$  s,  $b_2 = 36$  s,  $b_3 = 15$  s), the lower bound evaluates to  $\max(60, 0.4 + 60, 82.8 + 15) = 60.4$  s. Johnson’s Rule achieves 73.1 s in the schematic (21.0% above the schematic lower

bound); the gap arises because the 32B model’s 21.7 s load exceeds the 0.6B group’s 9 s compute window, leaving a 12.7 s residual that no ordering can eliminate. In the actual experiment, the JR ordering achieved 132.4 s (Table 4.4); the additional overhead relative to the schematic reflects vLLM initialisation costs ( $\sim 40\text{--}60$  s per transition) that are not modelled in the two-machine flow-shop formulation but remain constant across orderings and therefore do not affect the relative ranking among orderings.

## 5.5 Threats to Validity

**5.5.0.0.1 Internal Validity** Experiments were conducted on a shared cluster; other users’ processes and I/O could affect timing. Each data point is the mean of 3 independent runs with cold-start verification; standard deviations are reported.

Prefetching and Johnson’s Rule require diverse checkpoint sizes to produce meaningful variation; with similarly-sized models on fast SSD, I/O costs are small relative to compute and ordering has limited effect. Experiment 3 demonstrates scaling from 99 to 501 tasks with speedup growing from  $65\times$  to  $276\times$ ; larger-scale projections are indicative due to two-point calibration.

The FIFO extrapolation relies on cold-start sampling with explicit GPU release between tasks (6 tasks, 3 runs). The 32B cold-start showed substantial run-to-run variance ( $\text{CV} \approx 21\%$ ), likely due to non-deterministic TP=4 vLLM initialisation (NCCL setup, GPU memory allocation); the propagated  $\pm 3,353$  s uncertainty is reported in Table 4.1. The extrapolation is cross-validated against the independently measured 115.2 s transition gap (8B $\rightarrow$ 32B) from Experiment 2, confirming that 32B cold-load overhead dominates the per-switch cost. The explicit GPU-release protocol between tasks may slightly understate the true FIFO switch cost (omitting model unload time, typically 3–6 s per transition), making the reported extrapolation a conservative lower bound.

**5.5.0.0.2 External Validity** Results cover five Qwen models on one hardware configuration (RTX A5000, NVMe SSD) and may not generalise to mixture-of-experts architectures, other storage technologies (Lustre, CephFS), or larger clusters. Johnson’s Rule optimality holds for any  $k$  by theory; the experimental evaluation covers  $k = 3$  model groups, and generalisation to  $k > 3$  is left for future work. Faster storage would reduce the absolute benefit of grouping, though the relative advantage remains consistent.

**5.5.0.0.3 Construct Validity** Makespan does not capture per-task latency distribution, resource cost, or fairness; a multi-objective evaluation would be more appropriate for production. The flow-shop model also assumes no preemption and fixed group sizes; relaxing these would require open-shop or flexible flow-shop theory.

# Chapter 6

## Future Work

### 6.1 Inference-Level Extensions: Prefix-Aware Ordering and Multi-LoRA

The current system groups tasks by model but does not reorder tasks within a group by prompt prefix. vLLM’s built-in prefix caching already provides passive cache hits when prompts happen to share a prefix; actively sorting tasks within each group by prefix hash would maximise this reuse (e.g., RadixAttention in SGLang), with efficient prefix detection as the main challenge. The system also treats each LoRA adapter as a separate model; grouping by base model and using multi-LoRA serving in vLLM (Sheng et al., 2024) would give significant speedup for adapter-heavy workloads, since adapter swaps take  $< 1$ s versus 80–120s for a full reload.

### 6.2 Storage Extensions: Adaptive and Network-Aware Prefetching

The prefetch trigger currently fires at a fixed point; an adaptive policy observing task completion times in real time would maximise overlap for heterogeneous groups. For network storage, integrating with cloud APIs (S3, GCS, Azure Blob) and adjusting prefetch timing based on observed throughput would enable streaming from replicated storage without local caching, primarily an engineering effort given that Experiment 5 characterises the architecture analytically across storage tiers and validates the NVMe endpoint end-to-end.

### 6.3 Production Readiness: Scalability and Engine Abstraction

Three engineering improvements would enable production-scale deployment: streaming JSONL output to object storage, deadline-aware autoscaling ( $\text{workers} = \lceil n \cdot L_{\text{avg}} / T_{\text{deadline}} \rceil$ ),



and multi-node data parallelism with distributed queues. Abstracting the inference engine interface would add support for SGLang (Zheng et al., 2024) and TensorRT-LLM (NVIDIA, 2024b). The multi-GPU scheduling algorithm is implemented but not yet experimentally validated across configurations.

# Chapter 7

## Conclusion

This dissertation addressed makespan minimisation for LLM batch inference by formalising the problem as a two-machine flow-shop and proposing a shared-aware scheduling algorithm that applies Johnson’s Rule at the cluster scheduler level without modifying the inference engine.

### 7.1 Contributions

Five contributions are made: (1) a two-machine flow-shop formulation mapping model loading (I/O) and GPU inference (compute) to Johnson’s framework; (2) a shared-aware algorithm grouping tasks by model and applying Johnson’s Rule; (3) a checkpoint prefetching mechanism using the sllm-store pinned memory pool; (4) evaluation of prefetching across storage tiers (SSD, RAID, NFS); and (5) an OpenAI-compatible batch API extended for multi-model batches.

### 7.2 Key Findings

Model switching dominates multi-model batch execution, averaging  $\sim 90$  s per switch across the 0.6B–32B model range (measured from cold-start sampling). Model grouping eliminates  $O(n)$  switches, achieving an over-99% makespan reduction (extrapolated FIFO 45,495 s  $\rightarrow$  measured 228.5 s for 501 tasks,  $199\times$  speedup). Checkpoint prefetching hides the SSD I/O portion of model transitions, adding a further 9.5% improvement (228.5 s  $\rightarrow$  206.8 s). Johnson’s Rule ordering adds a further 20%, yielding a combined  $276\times$  speedup (164.7 s). The shared-aware strategy provides 65–276 $\times$  over FIFO across the measured range (99–501 tasks), with speedup growing as batch size increases. Prefetching hides 49.9% of cold-switch latency on local SSD (measured) and an analytically estimated 78.9% of I/O read time on 10 GbE NFS. Johnson’s Rule is robust to inference time uncertainty because LLM workloads are compute-heavy and the ordering depends only on checkpoint size. The algorithm operates above the inference engine, making it portable to vLLM, SGLang, TensorRT-LLM, and other backends; the OpenAI-compatible API enables drop-in migration from cloud batch services without client-side changes.

# Bibliography

- Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.
- Amazon Web Services. Amazon bedrock batch inference. <https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html>, 2024. Accessed: 2026-03-06.
- Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627. USENIX Association, 2017.
- Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153. USENIX Association, 2024.
- Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In

- 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, pages 323–336. USENIX Association, 2011.
- Ronald L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.
- Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.
- gRPC Authors. gRPC: A high performance, open source universal RPC framework. <https://grpc.io>, 2024. Accessed: 2026-03-06.
- Arpan Gujarati, Reza Karber, Safya Musavi, Antoine Peinado, Wolf Thelen, and Neeraja J. Yadwadkar. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, 2020.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- S. M. Johnson. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68, 1954.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023.
- Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679. USENIX Association, 2023.
- Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. CheckFreq: Frequent, fine-grained DNN checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 203–216. USENIX Association, 2021.
- NVIDIA. *CUDA C++ Programming Guide*, 2024a. Version 12.3. Accessed: 2026-03-06.
- NVIDIA. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference. <https://github.com/NVIDIA/TensorRT-LLM>, 2024b. Accessed: 2026-03-06.
- OpenAI. Openai batch api documentation. <https://platform.openai.com/docs/guides/batch>, 2024. Accessed: 2026-03-06.

- David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2013.
- Michael L. Pinedo. *Scheduling: Theory, algorithms, and systems*. 2016.
- Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 397–411. USENIX Association, 2021.
- Timos K. Sellis. Multiple-query optimization. *ACM Transactions on Database Systems*, 13(1):23–52, 1988.
- Mohammad Shahradd, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, 2020.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. *Proceedings of Machine Learning and Systems*, 6, 2024.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- The Kubernetes Authors. Kubernetes: Production-grade container orchestration. <https://kubernetes.io>, 2024. Accessed: 2026-03-06.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys ’15)*, pages 1–17. ACM, 2015.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In

- 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.
- Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*, 2010.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.